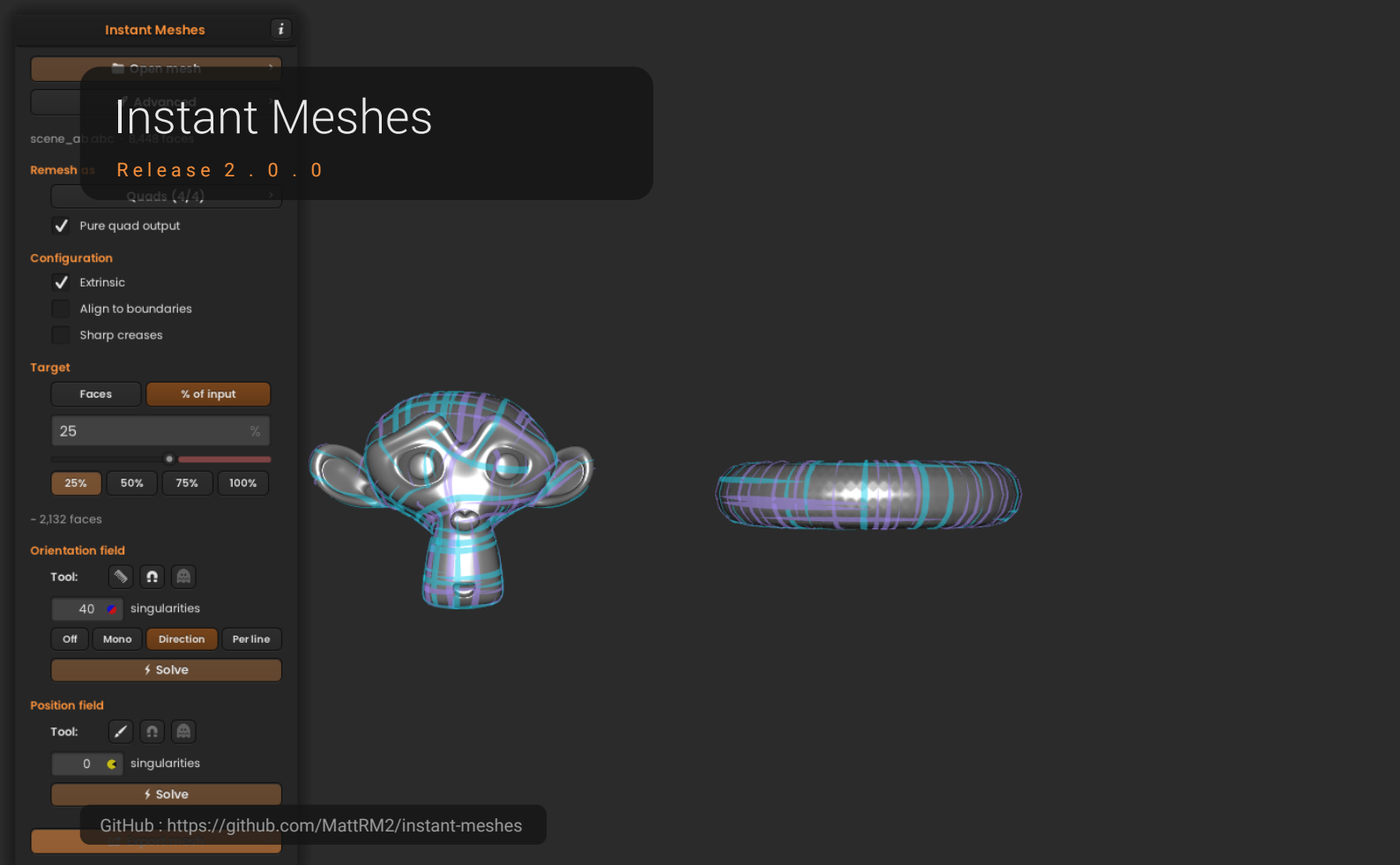




ALEMBIC · RETOPOLOGY · VFX PIPELINE

Instant Meshes

Documentation — MattRM2 fork of Instant Meshes 1.0

Field-aligned automatic retopology with native Alembic (.abc) support, per-mesh remeshing, percentage targets and the Matt Dark interface.

Created by Matthieu Barbié · 2026

Release 2.0.0

Based on Instant Meshes by W. Jakob, M. Tarini, D. Panozzo and O. Sorkine-Hornung (BSD license). Unofficial fork.

0 Table of Contents

1 Overview

- 1.1 What this fork adds
- 1.2 Installation
- 1.3 Files in the release

2 The Matt Dark Interface

- 2.1 Panel reference
- 2.2 Target: faces or percentage
- 2.3 Flow line colors
- 2.4 Typical workflow

3 Alembic Support

- 3.1 What is read
- 3.2 What is written
- 3.3 Robustness

4 Command Line

- 4.1 Batch mode
- 4.2 Options
- 4.3 Examples

5 Per-Mesh Remeshing (Alembic and OBJ)

- 5.1 Rules: -m and --others
- 5.2 Planning: --list and --dry-run
- 5.3 What happens to a remeshed mesh
- 5.4 OBJ scenes

6 Reference

- 6.1 Accuracy of the targets
- 6.2 Limitations
- 6.3 Credits and licenses

1 Overview

Automatic retopology, now at home in an Alembic pipeline

Instant Meshes turns dense or messy geometry (scans, sculpts, boolean results) into a clean quad or triangle mesh whose edges follow the shape of the model. This release is a fork of Instant Meshes 1.0 that keeps the original algorithm and its robustness, and adds what a VFX pipeline needs: **native Alembic** input and output, **per-mesh remeshing** inside an Alembic scene, **percentage targets**, and a restyled **Matt Dark** interface.

1.1 What this fork adds

- **Alembic (.abc)** read and written natively: no Alembic library, no dependency, polygon meshes, transforms and instances.
- **Per-mesh remeshing**: remesh MeshA at 75% and MeshB at 85% of their faces, keep the rest of the scene untouched, write one file (-m / -others), for **Alembic and OBJ** scenes.
- **Percentage targets**: 75% means 75% of the original face count, in the interface and on the command line.
- **Typed targets**: the tiny density slider is replaced by a text box (faces or %), presets and a full-width slider.
- **Meaningful colors**: flow lines by direction (U / V), your strokes in orange, singularities with consistent colors.
- **OBJ reader fixed**: n-gons (faces with more than 4 corners used to be cut silently) and negative (relative) indices.
- **InstantMeshes.exe**, without a space: easier to call from scripts.

1.2 Installation

- 1 Unzip the release anywhere, for example **D:\Tools\InstantMeshes**.
- 2 Run **InstantMeshes.exe**. Nothing to install: the executable is self-contained.
- 3 Optional: add the folder to your **PATH** to call **InstantMeshes** from any terminal or script.

1.3 Files in the release

File	Role
InstantMeshes.exe	The application: interface and command line
InstantMeshes_Documentation.pdf	This document
LICENSE.txt	Instant Meshes BSD license
OFL.txt	License of the Poppins typeface used by the interface

2

The Matt Dark Interface

Same workflow as Instant Meshes, clearer controls

The panel keeps the original top-to-bottom workflow: open a mesh, choose the output, set a target, solve the orientation field, solve the position field, export. Sections are titled in orange; the main actions (Solve, Extract, Export, Save) are orange buttons.

2.1

Panel reference

Control	What it does
Open mesh	Loads .obj, .ply, .abc or an .aln point cloud. The line below shows the file and its face count.
Advanced	Saves / loads the session, visualization modes, render layers, hierarchy level, crease angle.
Remesh as	Triangles (6/6), Quads 2/4 or Quads 4/4 (default).
Pure quad output	Subdivides the extracted mesh once so that every face is a quad. Moved here from the Export menu because the target accounts for it.
Configuration	Extrinsic smoothing (recommended), align to open boundaries, keep sharp creases above an angle.
Target	Output density, typed in faces or in % of the input (2.2).
Orientation field	Tools (comb, attractor, repeller), singularity count, flow line colors (2.3) and Solve.
Position field	Tools (edge brush, attractor, repeller), singularity count and Solve.
Export mesh	Smoothing iterations, Extract mesh, Show input / output, Save (.obj, .ply, .abc).

2.2

Target: faces or percentage

Choose **Faces** to type an exact face count, or **% of input** to type a percentage of the polygons of the loaded file (100% = as many faces as the input, 50% = half). Press **Enter** to apply. The line below the controls converts one into the other.

- **Presets** 25 / 50 / 75 / 100% apply a percentage in one click.
- The **slider** covers 1% to 400% on a logarithmic scale. Its **red range** marks targets the input is too coarse for: choosing one proposes to subdivide the input first.
- The target is the **final** face count: with **Pure quad output** the extraction aims at a quarter, the subdivision brings it back to the target.

Accuracy

On real meshes the result lands within about $\pm 3\%$ of the target. Very small meshes (a few hundred faces) can deviate more, since each closed object needs a minimum number of faces.

2.3 Flow line colors

Once the orientation field is solved, flow lines show how the edges of the output will run. Four modes, next to the Solve button:

Mode	Use it to
Off	See only your strokes and the singularities.
Mono	Read the flow without color noise (light grey).
Direction	Tell edge loops apart at a glance: cyan lines follow U, violet lines follow V (green: third direction in triangle mode). Default.
Per line	Follow one loop around the model: one color per line.

Orange is reserved for what you draw. Everything computed uses other colors:

	U flow lines	#22D3EE	Direction mode, first family
	V flow lines	#A78BFA	Direction mode, second family
	W flow lines	#4ADE80	Triangle mode, third family
	Comb strokes	#FB923C	Your orientation comb strokes
	Edge brush strokes	#F5F5F5	Your edge brush strokes
	Valence 3	#60A5FA	Orientation singularity
	Valence 5	#F87171	Orientation singularity
	Position singularity	#FACC15	Small offset
	Position singularity	#F472B6	Large offset
	Input mesh	#6B6B6B	Neutral clay so colors stand out

2.4 Typical workflow

- 1 **Open mesh** and pick **Remesh as** (Quads 4/4 for most models).
- 2 Set the **Target**, for example 50% of the input.
- 3 Click **Solve** under Orientation field. Check the flow in Direction mode; comb the field or move singularities where needed, then stop the solver.
- 4 Click **Solve** under Position field, then open **Export mesh**.
- 5 **Extract mesh**, check it with **Show output**, then **Save** as .obj, .ply or .abc.

3 Alembic Support

Native reader and writer, no Alembic library

Alembic files are read and written by code written for this fork, on top of the Ogawa container used by every current DCC. The files it writes open in Blender (and any Alembic reader) exactly like those written by the official library, down to the content hashes readers use for caching.

3.1 What is read

- Every **polygon mesh** of the file, placed in world space through its parent transforms (translate, rotate, scale, matrix, non-inheriting transforms).
- **Instances**: each instance appears at its own place.
- **N-gons** are triangulated for solving; quads keep the split of the OBJ reader, so an .abc and the same .obj give the same triangles.
- Animated files are read at their **first frame**.
- Cameras, curves, points and face sets are ignored; subdivision surfaces are skipped with a message.

3.2 What is written

- A new file: one transform and one polygon mesh named after the file, with quads and n-gons as extracted (no triangulation).
- In **per-mesh mode** (section 5): a copy of the input where only the chosen meshes change.
- The output may be the input file: it is fully read, then replaced in one atomic step. If anything fails, the original stays untouched.

3.3 Robustness

Every position, size and count read from a file is checked before use; nesting depth, cycles and instancing loops are bounded. A damaged file gives an error message, never a crash.

Tested

More than 31,000 deliberately corrupted files are read in the test suite without a single crash. Legacy HDF5-based Alembic files are refused with a clear message (re-export them with the Ogawa backend).

4

Command Line

Batch remeshing from scripts, farms and DCC tools

Give an output file with **-o** and Instant Meshes runs without a window: it loads, remeshes, writes and exits with code 0 on success. Without **-o** the interface opens on the file.

4.1

Batch mode

```
InstantMeshes.exe input.abc -o output.abc -f 75%
InstantMeshes.exe input.obj -o output.obj -f 5000 -D
```

4.2

Options

Option	Meaning
-o, --output	Output .obj, .ply or .abc (batch mode)
-f, --faces %	Face count, or percentage of the input polygons (75%)
-v, --vertices	Vertex count
-s, --scale	Edge length in world units
-D, --dominant	Quad-dominant output (no pure quad subdivision)
-r / -p	Orientation / position symmetry: 4/4 quads (default), 6/6 triangles
-c, --crease	Keep creases sharper than this angle
-b, --boundaries	Align to open boundaries
-S, --smooth	Smoothing iterations (default 2)
-d, --deterministic	Same result on every run (slower)
-t, --threads	Number of threads
-m, --mesh =	Per-mesh remeshing of an .abc or .obj (section 5)
--others	Target for every other mesh of the scene
--list / --dry-run	List the meshes / print the plan and stop
-V, --version	Print the version

4.3 Examples

```
# Half the faces, pure quads
InstantMeshes.exe scan.obj -o scan_retopo.obj -f 50%

# Triangles, exact count, sharp edges kept
InstantMeshes.exe part.abc -o part_tri.abc -r 6 -p 6 -f 20000 -c 30

# Replace the input file in place
InstantMeshes.exe scene.abc -o scene.abc -f 60%
```

Arguments are checked first

Unknown output formats, invalid percentages or conflicting options are reported before any computation, so a typo never costs minutes of remeshing.

5

Per-Mesh Remeshing (Alembic and OBJ)

Remesh some objects, keep the rest of the scene

With **-m** and **--others**, each chosen mesh of an Alembic or OBJ scene is remeshed on its own, with its own target, and written back into a copy of the scene, in the same format. Everything else, other objects, cameras, curves, animation and metadata, is copied untouched.

```
InstantMeshes.exe scene.abc -o scene_retopo.abc -m "MeshA=75%" -m "MeshB=85%" --others 25%
```

5.1 Rules: -m and --others

Rule	Selects
-m MeshA=75%	The object named MeshA, anywhere, at 75% of its own faces
-m Props/MeshA=5000	The object at that path from the root, 5000 faces
-m "Mesh*=60%"	Every object whose name starts with Mesh (* = any characters)
-m "Rock_??=50%"	Rock_01, Rock_AB... (? = exactly one character)
-m "Props/*=40%"	The children of Props, and the meshes below them
--others 25%	Every mesh no -m selects; without it they are copied untouched

- A name selects the object and every mesh below it; names are case-sensitive; the leading / is optional.
- When several -m select the same mesh, **the last one wins**: write the general rule first, then the exceptions.
- A rule that matches nothing is an error, found before any computation.
- Quote wildcards ("Mesh*=75%"): Git Bash, Linux and macOS shells would expand them.

5.2 Planning: --list and --dry-run

```
> InstantMeshes.exe scene.abc --list
Polygon meshes in "scene.abc": 2
  /Props/MeshA/MeshA      7872 faces    7958 vertices
  /Props/MeshB/MeshB      576 faces     576 vertices

> InstantMeshes.exe scene.abc -m "Mesh*=75%" -m MeshB=85% --dry-run
  /Props/MeshA/MeshA      7872 faces    -> 75% (~5904)    [-m Mesh*=75%]
  /Props/MeshB/MeshB      576 faces    -> 85% (~490)     [-m MeshB=85%]

Dry run: nothing computed, nothing written.
```

5.3 What happens to a remeshed mesh

Kept	Dropped
Name, place in the hierarchy, parent transforms (the mesh is remeshed in world space and written back in its own space)	UVs, normals and per-vertex / per-face attributes: they do not match the new topology
Object and user properties	Several face sets (per-face materials): they cannot follow the new faces
A single face set (one material), rebuilt over all new faces	

Refused on purpose

Animated meshes and instanced meshes cannot be targets (-m reports them; --others copies them untouched and says so in the plan). If one mesh fails, nothing is written.

5.4 OBJ scenes

```
InstantMeshes.exe scene.obj --list
InstantMeshes.exe scene.obj -o scene_retopo.obj -m "MeshA=50%" --others 80%
```

- **Objects** are the **o** blocks of the file, or its **g** groups when it has no **o** line (Maya, ZBrush). --list shows the names found; paths are /Name.
- Untouched objects keep their lines as they are: positions, UVs, normals, materials, groups and comments. Only their face indices are renumbered, since the replaced objects change the vertex count.
- A replaced object keeps its **most used material**: OBJ faces inherit the last usemtl, so an object cannot be left without one. Its UVs, normals and smoothing groups are dropped.
- Vertices shared with an untouched object are kept; the .mtl file is not touched.
- The output must be an .obj; it may be the input file (atomic replacement).

6 Reference

6.1 Accuracy of the targets

Input	Faces	Deviation measured (25 to 150%, quad and -D modes)
Two-object scene	8,448	-1.8% to +1.6%
Open mesh	1,968	-2.6% to +2.0%
Tiny meshes	86	up to $\pm 33\%$

6.2 Limitations

- Alembic: polygon meshes only; subdivision surfaces, curves and points are not remeshed; animation is read at the first frame.
- Remeshed meshes lose their UVs and normals (new topology).
- Point clouds (.aln) accept face counts, not percentages.
- The per-mesh mode (-m / -others, Alembic and OBJ) is available on the command line; the interface remeshes the whole loaded file.

6.3 Credits and licenses

Instant Meshes: Wenzel Jakob, Marco Tarini, Daniele Panozzo, Olga Sorkine-Hornung, Instant Field-Aligned Meshes, ACM Transactions on Graphics (SIGGRAPH Asia 2015). BSD license, see LICENSE.txt.

This fork (Alembic support, per-mesh remeshing, percentage targets, Matt Dark interface): Matthieu Barbi , 2026, same license. Source: <https://github.com/MattRM2/instant-meshes>

Poppins typeface (interface): The Poppins Project Authors, SIL Open Font License 1.1, see OFL.txt. Hash algorithms: MurmurHash3 (Austin Appleby) and SpookyHash (Bob Jenkins), public domain.